



Prova (a₂)

3/10/2025

Nome: _____

Matrícula: _____

1 Instruções

1. Para fazer essa prova, você deve clonar o repositório no seu computador, resolver as questões abaixo e empurrar as soluções para o repositório remoto.
2. Ao clonar o repositório, é preciso instalar as dependências com o comando `npm install`.
3. Cada questão deve ser resolvida no arquivo correspondente, que está localizado no diretório `/questoes`.
4. Apenas os arquivos do diretório `/questoes` podem ser alterados. Não é permitido alterar outros arquivos ou diretórios.
5. O repositório já está devidamente configurado e não é necessário fazer nenhuma configuração adicional.
6. Você pode testar suas soluções localmente utilizando o comando `npm run test`. Esse comando irá compilar os arquivos TypeScript e executar os testes automatizados.
7. Caso queira executar os testes de uma questão específica, utilize o comando `npm run test:X`, onde `X` é o número da questão (1, 2, 3 ou 4).
8. A nota máxima da prova é 10 pontos.
9. A prova contém 4 questões, totalizando 13 pontos. Você pode escolher quais questões deseja resolver, mas a nota da prova não ultrapassará 10 pontos.
10. É permitida a consulta somente ao material que está na pasta `/docs` e ao repositório dos exercícios de laboratório. Não é permitida a consulta a qualquer outro material, incluindo a internet.
11. A alteração de qualquer arquivo fora do diretório `/questoes` resultará em nota zero.
12. A detecção de plágio resultará em nota zero para todos os envolvidos.

2 Questões

1. (2 pontos) Utilize **programação funcional em JavaScript** para implementar um sistema simples de gerenciamento de produtos em um estoque. Considere que cada produto tem o formato `{ id: number, nome: string, categoria: string, preco: number, qtd: number }`. O sistema deve ter as seguintes funcionalidades:
 - a) Filtrar por categoria:
 - nome da função: `produtosPorCategoria`
 - parâmetros: `produtos` (array de produtos), `categoria` (string)
 - retorno: novo array contendo apenas os produtos da categoria informada
 - restrições:
 - usar `filter`;
 - retornar um array vazio se `produtos` estiver vazio ou se não houver produtos na categoria;

- não usar laços imperativos;
 - não alterar o array de entrada.
- b) Listar nomes formatados
- nome da função: `nomesFormatados`
 - parâmetros: `produtos` (array de produtos)
 - retorno: novo array de strings no formato "ID: [id] | Nome: [nome] | Categoria: [categoria]"
 - restrições:
 - usar `map` e *template literals*;
 - retornar um array vazio se `produtos` estiver vazio;
 - não usar laços imperativos;
 - não alterar o array de entrada.
- c) Total de estoque (valor monetário)
- nome da função: `totalEstoque`
 - parâmetros: `produtos` (array de produtos)
 - retorno: número representando a soma de `preco * qtd` de todos os produtos
 - restrições:
 - usar `reduce`;
 - retornar 0 para array vazio;
 - não usar laços imperativos;
 - não alterar o array de entrada.
- d) Média de preço por categoria
- nome da função: `mediaPrecoPorCategoria`
 - parâmetros: `produtos` (array de produtos), `categoria` (string)
 - retorno: número com a média de `preco` dos produtos da categoria (ou 0 se não houver itens)
 - restrições:
 - retornar 0 se `produtos` estiver vazio ou se não houver produtos na categoria;
 - usar `filter` e `reduce`;
 - não usar laços imperativos;
 - não alterar o array de entrada.

2. (2 pontos) Utilize **orientação a objetos** em **TypeScript** para modelar um sistema bancário simples seguindo os requisitos abaixo:

- a) Uma classe abstrata `ContaBancaria`
- propriedades (protegidas):
 - `saldo` (number)
 - `titular` (string)
 - construtor: deve inicializar as propriedades `saldo` e `titular`
 - métodos:
 - `depositar`
 - * parâmetro: `valor` (number)
 - * retorno: nenhum
 - * comportamento: adiciona o valor ao saldo
 - `sacar` (abstrato)
 - * parâmetro: `valor` (number)
 - * retorno: boolean
 - `exibirSaldo`
 - * parâmetro: nenhum
 - * retorno: string no formato "Titular: [titular], Saldo: [saldo]"
- b) A classe `ContaCorrente` que estende `ContaBancaria`

- propriedade (privada):
 - `limiteChequeEspecial` (number)
- construtor: deve inicializar as propriedades herdadas e a propriedade `limiteChequeEspecial`
- métodos:
 - implementar o método `sacar`
 - * comportamento: permite sacar se o valor for menor ou igual ao saldo mais o limite do cheque especial, de \$ 500,00;
 - * retorno: do tipo booleano para indicar se a operação foi bem sucedida ou não.

c) A classe `ContaPoupanca` que estende `ContaBancaria`

- métodos:
 - implementar o método `sacar`
 - * comportamento: permite sacar se o valor for menor ou igual ao saldo;
 - * retorno: do tipo booleano para indicar se a operação foi bem sucedida ou não.

d) A interface `Banco` para permitir adicionar múltiplas contas bancárias. A chave deve ser o CPF do titular e o valor deve ser uma instância de `ContaBancaria`.

3. (4 pontos) Implemente em **JavaScript** uma pequena agenda de contatos com os seguintes requisitos:

a) Adicionar contato:

- nome da função: `adicionarContato`
- parâmetro: um objeto com as propriedades `id`, `nome` e `telefone`
- retorno: do tipo `boolean` para indicar sucesso ou falha
- restrições:
 - contatos com `id` duplicado não podem ser adicionados;
 - todas as propriedades do contato são obrigatórias;
 - a função deve utilizar o método `push()` para adicionar o contato à lista de contatos.

b) Remover contato:

- nome da função: `removerContato`
- parâmetro: um `id` de contato
- retorno: do tipo `boolean` para indicar sucesso ou falha
- restrições:
 - o contato deve existir para ser removido;
 - a função deve utilizar o método `filter()` para remover o contato da lista de contatos.

c) Buscar contato:

- nome da função: `buscarContato`
- parâmetro: um nome de contato
- retorno: o objeto do contato encontrado ou `null` se não encontrado
- restrições:
 - o nome a ser buscado pode estar em letras maiúsculas ou minúsculas;
 - a função deve utilizar o método `find()` para buscar o contato na lista de contatos.

d) Listar contatos:

- nome da função: `listarContatos`
- parâmetro: nenhum
- retorno: uma lista com os contatos transformados em strings no formato "ID: [id], Nome: [nome], Telefone: [telefone]"
- restrições:
 - a função deve retornar um array vazio se não houver contatos na lista.
 - a função deve utilizar o método `map()`;
 - a função deve utilizar `template literals` para formatar as strings dos contatos.

e) Limpar agenda:

- nome da função: `limparAgenda`
- parâmetro: nenhum
- retorno: nenhum
- comportamento: remove todos os contatos da lista de contatos

4. (5 pontos) Pilhas são estruturas de dados que permitem armazenar e gerenciar elementos de forma ordenada, seguindo o princípio LIFO (Last In, First Out). Elas são amplamente utilizadas em diversas aplicações, como gerenciamento de histórico de navegação, chamadas de funções, avaliação de expressões matemáticas, entre outras. Suas operações principais incluem `empilhar` (`push`), que adiciona um elemento ao topo da pilha, e `desempilhar` (`pop`), que remove o elemento do topo da pilha.

Utilizando pilhas, implemente em **TypeScript** o comportamento de um pequeno editor de texto no navegador que permita ao usuário digitar texto e realizar as operações de desfazer e refazer. O editor deve ter as seguintes funcionalidades:

- Digitado texto: o usuário deve poder inserir texto na área de edição.
- Desfazer (Undo): o usuário deve poder reverter a última alteração feita.
- Refazer (Redo): o usuário deve poder reaplicar a última alteração que foi desfeita.

Além disso, o editor deve atender aos seguintes requisitos:

- Estrutura e tipagem:
 - Os estados devem ser armazenados em duas pilhas: uma para desfazer e outra para refazer.
 - Os pilhas devem permitir porlimorfismo, ou seja, devem ser capazes de armazenar qualquer tipo de dado.
- DOM e layout:
 - Os botões de `Desfazer` e `Refazer` devem estar desabilitados quando não houver ações para desfazer ou refazer, respectivamente.
 - Quando o usuário digitar seu texto, o estado atual do editor deve ser salvo na pilha de desfazer.
 - Ao clicar em `Desfazer`, o estado atual deve ser movido para a pilha de refazer e o estado anterior deve ser restaurado no editor.
 - Ao clicar em `Refazer`, o estado atual deve ser movido para a pilha de desfazer e o estado mais recente da pilha de refazer deve ser restaurado no editor.
 - Quando o usuário digitar um novo texto após desfazer uma ação, a pilha de refazer deve ser esvaziada.
 - Utilize contadores para exibir a quantidade de estados armazenados em cada pilha.
 - Além dos botões, o usuário deve poder utilizar os atalhos de teclado `Ctrl + Z` para desfazer e `Ctrl + Y` para refazer.

Os arquivos `index.html` e `styles.css` já estão prontos e não precisam ser alterados. Você deve focar apenas na implementação do arquivo `questao4.ts`.